Fibonacci heaps solutions

19-E01 Extract-min root list

Derivation and check

Removing2 promotes7,5 to roots with parent=NIL and mark=false. Their degrees0 collide,so link larger7 under5 and make5.degree=1. Consolidation rebuilds roots and minimum among survivors;it need not claim a minimum scan before merging. Final root/min is5,with child7. No other roots existed in this stated heap.

19-E02 Cascading cut

Derivation and check

Cut x from p,update its child ring/degree,and set x.parent=NIL,x.mark=false. Since marked p has lost another child,cut p too and clear its mark. If unmarked g is nonroot,mark g and stop;it remains under its old parent. If g is root,stop and retain mark=false. Both branches add x,p to existing roots,with their parents NIL. Do not conflate g’ s different mark outcomes;higher links/marks remain unchanged.

19-E03 Minimum subtree by degree

Derivation and check

Degree5 gives size≥F7. F3=2,F4=3,F5=5,F6=8,F7=13,so at least13 nodes. This recursive minimum size reflects allowed child-loss histories and grows as Fibonacci numbers. Inverting exponential growth gives maximum degree O(log n).

19-E04 Total cascade account

Derivation and check

First x is cut because it becomes smaller than its parent and need not be marked. The later c−1 cascaded nodes were marked and each clears one mark. At most one final nonroot gains a mark,so $\Delta m \le 2-c$,$\Delta t=c$,and $\Delta(t+2m) \le 4-c$. Choose $\Psi=C(t+2m)$,$C \ge a$,then$\hat{c} \le ac+b+C(4-c) \le b+4C$. Choose the same fixed C large enough for extract-min pointer constants too. Nonnegative Ψ and initial0 telescope without any distributional assumption on c.

Fibonacci heaps solutions (continued)

19-E05 A binary degree-array bound is insufficient

Derivation and check

Lowering id16 cuts it to roots. Its previously unmarked parent id15 gains a mark and is not cut. Extracting id16 leaves root1 with degree4 and15 total nodes. There is only one root,so consolidation does not change its degree. floor($\log_2 15$)=3,so array0..3 cannot store degree4. This is a legal-history counterexample. The Fibonacci bound requires degree-four subtree$\geq$F6=8,not2^4=16. Use floor(log_ϕ n) conservatively or expand slots on demand.

19-E06 Erase a specified handle among equal minima

Derivation and check

Another y may already have INT_MIN. Lowering x to the same value need not make min point to x,so extract-min can remove y instead. Abstract$-\infty$ must be strictly below every ordinary key. If x is nonroot,cut it and cascade from its old parent. Remove exactly root x,promote and unmark its children,consolidate,and recompute min. Only x’ s handle becomes invalid;other equal-key identities remain. Cuts cost O(1) amortized and root removal O(log(n+1)) by the same degree/root-count potential argument.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Fibonacci heaps solutions (continued)

Verification code appendix: cascade_cut_state.py

```
"""Evaluate both cascade-cut branches without merging their conclusions."""

from dataclasses import dataclass


@dataclass
class Node:
    name: str
    parent: object = None
    marked: bool = False


def cascade(root_is_g):
    root_list = []
    x, p, g = Node("x"), Node("p", marked=True), Node("g")
    root_list += [x, p]

    for node in (x, p):
        node.parent = None
        node.marked = False

    if not root_is_g:
        g.marked = True
    return root_list, {node.name: node.marked for node in (x, p, g)}


def run():
    nonroot_list, nonroot_marks = cascade(False)
    root_list, root_marks = cascade(True)
    assert [node.name for node in nonroot_list] == ["x", "p"]
    assert nonroot_marks == {"x": False, "p": False, "g": True}
    assert [node.name for node in root_list] == ["x", "p"]
    assert root_marks == {"x": False, "p": False, "g": False}
    print("PASS cascade nonroot_g=True root_g=False")
    return nonroot_marks, root_marks


if __name__ == "__main__":
    run()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Fibonacci heaps solutions (continued)

Verification code appendix: fibonacci_checks.py

```
"""Logical Fibonacci-heap state model; Python lists do not model pointer costs."""
from dataclasses import dataclass, field
from random import Random


@dataclass(eq=False)
class Node:
    identity:int
    key:int
    parent:object=None
    children:list=field(default_factory=list)
    mark:bool=False


class Heap:
    def __init__(self):
        self.roots=[]
        self.nodes={}
        self.minimum=None
        self.next_id=0
        self.events=set()

    def insert(self,key):
        x=Node(self.next_id,key)
        self.next_id+=1
        self.nodes[x.identity]=x
        self.roots.append(x)
        if self.minimum is None or x.key<self.minimum.key:
            self.minimum=x
        return x

    def consolidate(self):
        degrees={}
        for x in list(self.roots):
            d=len(x.children)
            while d in degrees:
                y=degrees.pop(d)
                if y.key<x.key:
                    x,y=y,x
                y.parent=x
                y.mark=False
                x.children.append(y)
                d+=1
            degrees[d]=x
        self.roots=list(degrees.values())
        self.minimum=min(self.roots,key=lambda x:x.key,default=None)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci heaps solutions (continued)

```
    def cut(self,x,parent):
        parent.children.remove(x)
        x.parent=None
        x.mark=False
        self.roots.append(x)
        self.events.add("cut")

    def cascading_cut(self,y):
        parent=y.parent
        if parent is not None:
            if not y.mark:
                y.mark=True
                self.events.add("mark-nonroot")
            else:
                self.cut(y,parent)
                self.events.add("cascade")
                self.cascading_cut(parent)
        else:
            self.events.add("stop-at-root")

    def decrease(self,x,new_key):
        assert new_key<=x.key and self.nodes[x.identity] is x
        x.key=new_key
        parent=x.parent
        if parent is not None and x.key<parent.key:
            self.cut(x,parent)
            self.cascading_cut(parent)
        if self.minimum is None or x.key<self.minimum.key:
            self.minimum=x

    def extract_root(self,x):
        assert x.parent is None
        self.roots.remove(x)
        for child in x.children:
            child.parent=None
            child.mark=False
            self.roots.append(child)
        del self.nodes[x.identity]
        self.consolidate()
        return x.key

    def pop_min(self):
        if self.minimum is None:
            return None
        return self.extract_root(self.minimum)

    def erase(self,x):
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Fibonacci heaps solutions (continued)

```
        parent=x.parent
        if parent is not None:
            self.cut(x,parent)
            self.cascading_cut(parent)
        return self.extract_root(x)

    def potential(self):
        return len(self.roots)+2*sum(x.mark for x in self.nodes.values())

    def validate(self):
        seen=set()
        def visit(x,parent):
            assert x.identity not in seen
            seen.add(x.identity)
            assert self.nodes[x.identity] is x and x.parent is parent
            assert parent is None or parent.key<=x.key
            assert parent is not None or not x.mark
            size=1+sum(visit(child,x) for child in x.children)
            f0,f1=0,1
            for _ in range(len(x.children)+2):
                f0,f1=f1,f0+f1
            assert size>=f0
            return size
        for root in self.roots:
            visit(root,None)
        assert seen==set(self.nodes)
        assert self.minimum is None if not self.nodes else self.minimum.key==min(x.key for x
in self.nodes.values())
        return sorted(x.key for x in self.nodes.values())


def cascade_example(nonroot_g):
    heap=Heap()
    for key in range(65):
        heap.insert(key)
    assert heap.pop_min()==0
    target=next(x for x in heap.nodes.values()
                if len(x.children)>=2 and x.parent is not None
                and (x.parent.parent is not None)==nonroot_g)
    a,b=target.children[:2]
    g=target.parent
    ids=(a.identity,b.identity,target.identity,g.identity)
    heap.decrease(a,-10)
    assert target.mark
    before=heap.potential()
    heap.decrease(b,-11)
    assert target.parent is None and not target.mark
    assert g.mark==nonroot_g
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Fibonacci heaps solutions (continued)

```
    heap.validate()
    return ids,before,heap.potential()


def main():
    heap=Heap()
    for i in range(9):
        heap.insert(i)
    heap.pop_min()
    assert len(heap.roots)==1
    assert heap.roots[0].key==1 and len(heap.roots[0].children)==3
    assert heap.validate()==list(range(1,9))
    degree_counterexample=Heap()
    handles=[degree_counterexample.insert(i) for i in range(17)]
    assert degree_counterexample.pop_min()==0
    degree_counterexample.decrease(handles[16],-1)
    assert degree_counterexample.pop_min()==-1
    assert len(degree_counterexample.nodes)==15
    assert len(degree_counterexample.roots[0].children)==4
    degree_counterexample.validate()
    examples=[cascade_example(False),cascade_example(True)]
    rng=Random(1920)
    heap=Heap()
    operations=0
    for _ in range(2400):
        action=rng.randrange(4)
        if action==0 or not heap.nodes:
            heap.insert(rng.randrange(-200,201))
        elif action==1:
            x=rng.choice(list(heap.nodes.values()))
            heap.decrease(x,x.key-rng.randrange(50))
        elif action==2:
            expected=min(x.key for x in heap.nodes.values())
            assert heap.pop_min()==expected
        else:
            x=rng.choice(list(heap.nodes.values()))
            old=x.key
            assert heap.erase(x)==old and x.identity not in heap.nodes
        heap.validate()
        operations+=1
    print(f"PASS Fibonacci_logical_operations={operations} consolidation_root1_degree3
n15_degree4_counterexample=True cascade_examples={examples} events={sorted(heap.events)}")


if __name__=="__main__":
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Fibonacci heaps solutions (continued)

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study